Machine Coding: File Upload Preview

Problem: Build a file picker that lets the user choose or drag & drop files, validates them, and shows a preview (thumbnail / video / PDF link) of each file with its name and size, before uploading them.

Tech stack: React 19 (Vite, JavaScript). The project has no Tailwind installed, so styling is plain CSS.

Time limit: 60 minutes

PART A: Thought process

Every machine coding round can be solved with the same loop:

1. **Understand** what is asked: functional + non-functional requirements, ask clarifying questions, write down assumptions.
2. **List the interactions** (every action the user can take and what should happen). This list is your checklist.
3. **Design components** (who renders what, who owns what).
4. **Design state**: keep the minimum, derive the rest.
5. **Define data shapes and contracts** (what goes in, what comes out, API mocks).
6. **Trace one flow end to end** to prove the design works.
7. **Spot the hard parts early** and decide how to solve them before coding.
8. **Build in layers**: smallest working version first, then add one layer at a time, testing each.
9. **Cover states and edge cases**: empty, loading, error, success, limits.
10. **Wrap up**: explain trade-offs and what you would add with more time.

Time budget (60 minutes)

Phase	Time
Requirements + clarifying questions	5 min
Interactions, components, state, contracts	8 min
Core build (picker, validation, preview grid)	25 min
Drag & drop, remove, upload mock, edge cases	12 min
Styling, accessibility pass, manual testing	7 min
Wrap-up (trade-offs, what next)	3 min

> **Think out loud.** The interviewer grades your reasoning, not just the final UI. Say *why* you pick `URL.createObjectURL` over `FileReader`, *why* validation is a pure function, *why* you reset the input value. Silence looks like being stuck.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Pick one or many files with a **Browse** button (native file picker).
- **Drag & drop** files onto a drop zone, with a visual "drag over" state.
- **Validate** each file: allowed type (images, videos, PDF), max size (10 MB), max count (6), no duplicates, no empty files.
- Show **invalid files with a reason**; valid files in the same batch are still added.
- Show a **preview card** per file: image thumbnail, playable video, or a PDF "open" link, plus name and readable size.
- **Remove** a single file, **Clear all**.
- Show **count and total size**.
- **Upload** the selected files (mocked), with uploading / success / error states.

Non-functional requirements

- **Performance:** previews must be instant even for a 10 MB video; no base64 copies of big files in memory.
- **Memory safety:** every preview URL created must be released (no blob leaks when removing files or leaving the page).
- **Error handling:** bad files never break the list; a failed upload keeps the files so the user can retry.
- **Accessibility:** the drop zone has a real button (keyboard + screen reader), the remove button has a label (`Remove photo.png`), errors use `role="alert"` , status uses `aria-live` .
- **Reusability:** validation rules live in one `RULES` object; validation is a pure function; selection logic is a custom hook.
- **Extensibility:** adding a new preview kind (audio, text) is one branch in `Media` ; adding progress/retry per file is a new field on each item.

Clarifying questions to ask

1. Which file types are allowed, and what is the max size and max count?
2. Should invalid files block the whole batch, or should valid ones still be added?
3. Is selecting the same file twice a duplicate (ignore) or allowed?
4. Do we upload immediately on select, or only when the user clicks Upload?
5. Is there a real API? Do we need per-file progress and retry? (Usually a separate task.)
6. Which preview is expected for non-images (video player, PDF embed, icon)?

Assumptions we build with

- Allowed: `image/*` , `video/*` , `application/pdf` . Max **10 MB** per file, max **6 files** total.
- **Partial accept:** valid files are added, invalid ones are listed with a reason.
- A duplicate = same `name + size + lastModified` ; it is rejected with "already added".

- Upload happens only on click, all files in one request; the API is **mocked** (1.2 s delay, 15% random failure).
- PDF preview is a link that opens the file in a new tab (embedding many PDFs is heavy).
- No per-file progress / retry (that is task _41).

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Drop zone + hint ("Images, videos or PDF · up to 10.0 MB each · max 6 files"), empty message "No files selected yet."
Click **Browse files**	Native picker opens, filtered to `image/*,video/*,application/pdf`
Pick 2 valid images	2 preview cards appear with thumbnails, name, size; toolbar shows "2 files · 3.5 KB"
Drag files over the zone	Zone border turns blue (active state), and does not flicker when moving over its children
Drop files on the zone	Same as picking; active state removed; browser does **not** open the file
Drop a `.exe` with a valid image	Image added; error "virus.exe: file type not allowed"
Pick a 25 MB video	Error "big.mp4: larger than 10.0 MB"
Pick a 0-byte file	Error "empty.png: file is empty"
Pick the same file again	Error "photo.png: already added" (the change event still fires because the input was reset)
Add files beyond 6	First ones fill up to 6, rest get "limit of 6 files reached"
Click × on a card	Card removed, its object URL revoked
Click **Clear all**	All cards and errors removed, all URLs revoked
Click **Upload N files**	Button shows "Uploading…", drop zone / remove / clear disabled
Upload succeeds	List cleared, "Upload complete." announced
Upload fails	Files kept, "Upload failed. Please try again." announced
Add files after a success/fail	Old status message cleared
Keyboard user	Tab to Browse, Enter opens picker; Tab to each remove button (labelled)

This table is the implementation and test checklist.

---

3. Design the architecture

Module	Responsibility
`App`	Container: wires the hook, owns upload status, renders errors, grid, toolbar, upload button

Module	Responsibility
`Dropzone`	Drag & drop + hidden file input; hands a list of `File`s to `onFiles`
`FilePreview`	One card: creates/revokes the object URL, renders media, name, size, remove button
`useFileSelection`	Hook: owns the selected items + last errors; `addFiles`, `removeFile`, `clearAll`
`fileUtils`	Pure helpers: `RULES`, `validateFiles`, `formatBytes`, `getKind`, `fileKey`
`uploadFiles`	API layer (mock): builds `FormData`, returns a promise

```
App  (status: idle | uploading | success | error)
├── useFileSelection()  → items, errors, addFiles, removeFile, clearAll
│     └── validateFiles()  (pure, fileUtils)
├── Dropzone  (onFiles, disabled)
│     └── <input type="file" hidden>
├── errors list  (role="alert")
├── toolbar  (count · total size, Clear all)
├── <ul> FilePreview × N  (item, onRemove, disabled)
│           └── Media (img | video | pdf link | icon)
└── Upload button → uploadFiles()  (api)
```

Start with everything in `App`; split `Dropzone` and `FilePreview` out as soon as they have their own state (drag state, object URL). The hook and the pure validator come out next because they are what you want to unit test.

---

4. Design the state / data model

State or field	Owner	Purpose
`items: [{ id, file, kind }]`	`useFileSelection`	The selected files, in order
`errors: string[]`	`useFileSelection`	Messages for the last add (replaced on each add)
`status: 'idle' | 'uploading' | 'success' | 'error'`	`App`	Upload lifecycle; one enum instead of 3 booleans
`isDragging: boolean`	`Dropzone`	Visual drag-over state
`dragDepth` (ref)	`Dropzone`	Counts nested dragenter/dragleave; no re-render needed
`url`	`FilePreview`	Object URL for this card's file

```
const [items, setItems] = useState([]);    // [{ id, file, kind }]
const [errors, setErrors] = useState([]);  // ['virus.exe: file type not allowed']
const [status, setStatus] = useState('idle'); // idle | uploading | success | error
```

Senior notes

- **Derived, not stored:** total size (`items.reduce`), count, `isUploading = status === 'uploading'` , the "empty" state (`items.length === 0`). Storing them would let them go out of sync.
- **Keep the real `File` object** in state. It is the only thing the upload needs, and the browser does not copy the bytes; it is a handle.
- **Object URL is owned by the card**, not by the list. That ties its lifetime to the card's mount/unmount (see challenge 1).
- `dragDepth` **is a ref**, because changing it should not re-render.
- `status` lives in `App` (not the hook) because selection and upload are separate concerns; the hook can be reused in a form that uploads differently.

---

5. Design the data structures and contracts

Example data

```
// one item in state
{
  id: '2f6c…',          // crypto.randomUUID(): stable React key, survives removals
  file: File {          // native File from <input> or DataTransfer
    name: 'photo.png',
    size: 3584,         // bytes
    type: 'image/png',  // MIME; can be '' for unknown extensions
    lastModified: 1730…,
  },
  kind: 'image',        // 'image' | 'video' | 'pdf' | 'file' (derived once from type)
}
```

Why a generated `id` and not the index? Removing card 2 would shift every index, and React would reuse the wrong `<video>` / `<img>` element. Why not `file.name` ? Two different files can have the same name.

Contracts

```
// pure validator
validateFiles(incoming: File[], existing: File[], rules = RULES)
  → { accepted: File[], errors: string[] }

// rules (single source of truth for picker filter, hint text and validation)
RULES = { accept: ['image/', 'video/', 'application/pdf'], maxSize: 10 * 1024 * 1024, maxFiles: 6 }

// hook
useFileSelection(rules?) → { items, errors, addFiles(FileList | File[]), removeFile(id), clearAll() }

// components
<Dropzone onFiles={(files) => …} disabled={bool} />
<FilePreview item={item} onRemove={(id) => …} disabled={bool} />
```

API contract (mocked)

```
POST /api/upload
Content-Type: multipart/form-data; boundary=…   (set by the browser)
body: files=<photo.png>, files=<resume.pdf>
```

```
200 → { "uploaded": 2 }
5xx / network error → promise rejects → status 'error', files kept
```

6. Trace the data flow

User drops `photo.png` and `virus.exe` on the zone:

```
drop event on .dropzone
  ↓  e.preventDefault()  (stop the browser from opening the file)
  ↓  dragDepth = 0, isDragging = false
Dropzone → onFiles(e.dataTransfer.files)
  ↓
App.handleFiles → setStatus('idle'), addFiles(files)
  ↓
useFileSelection.addFiles
  ↓  validateFiles([photo, virus], existingFiles, RULES)
  ↓     photo.png  → type ok, size ok, not seen, under limit → accepted
  ↓     virus.exe  → type not allowed                        → error
  ↓  setErrors(['virus.exe: file type not allowed'])
  ↓  setItems(prev => [...prev, { id: uuid(), file: photo, kind: 'image' }])
  ↓
re-render
  ↓  errors box (role="alert") announces the error
  ↓  new <FilePreview> mounts
FilePreview useEffect
  ↓  url = URL.createObjectURL(photo)   → 'blob:http://localhost/…'
  ↓  setUrl(url) → <img src="blob:…">   (instant, no file read)
  ↓
Output: card with thumbnail, "photo.png", "3.5 KB"; toolbar "1 file · 3.5 KB"

Later: user clicks × → removeFile(id) → card unmounts → effect cleanup → URL.revokeObjectURL(url)
```

7. Important technical challenges

Challenge 1: Showing a preview without leaking memory

Problem. The user picks a 10 MB video. Reading it with `FileReader.readAsDataURL` creates a ~13 MB base64 string (33% bigger), takes time, and lives in JS memory. `URL.createObjectURL(file)` is instant (it is just a pointer to the file), **but** the browser keeps the file alive until you call `URL.revokeObjectURL`. If you add and remove 50 files, or navigate away, you leak 50 blobs.

Where to create it?

- In `addFiles` and store in state → you must remember to revoke in `removeFile`, `clearAll` and on unmount (3 places, easy to miss one).
- In `useMemo` → no cleanup hook, and StrictMode may call it twice (one URL leaks).

Solution. Create the URL in the card's effect and revoke it in the cleanup. Lifetime of the URL = lifetime of the card. Remove, Clear all, successful upload and page unmount are all covered automatically. StrictMode's mount → unmount → mount simply creates, revokes, creates again.

```
const [url, setUrl] = useState(null);

useEffect(() => {
  const objectUrl = URL.createObjectURL(file);
  setUrl(objectUrl);
  return () => URL.revokeObjectURL(objectUrl);
}, [file]);
```

Challenge 2: Validation you can trust (and test)

Problem. `accept="image/*"` on the input is only a **hint**: the user can switch the picker to "All files", and drag & drop ignores `accept` completely. Also: a batch can mix good and bad files, the count limit depends on files already selected, and the same file can appear twice inside one batch.

Solution. One pure function, driven by one `RULES` object (the same object also builds the `accept` attribute and the hint text, so they never disagree). Check in order: type → empty → size → duplicate → limit. Track `seen` keys inside the loop so duplicates within the same batch are caught too. Return accepted files + human-readable errors; the hook just applies the result.

```
export function validateFiles(incoming, existing, rules = RULES) {
  const seen = new Set(existing.map(fileKey));
  const accepted = [];
  const errors = [];
  for (const file of incoming) {
    const key = fileKey(file);
    if (!isAccepted(file.type, rules.accept)) errors.push(`${file.name}: file type not allowed`);
    else if (file.size === 0) errors.push(`${file.name}: file is empty`);
    else if (file.size > rules.maxSize) errors.push(`${file.name}: larger than
${formatBytes(rules.maxSize)}`);
    else if (seen.has(key)) errors.push(`${file.name}: already added`);
    else if (existing.length + accepted.length >= rules.maxFiles) errors.push(`${file.name}: limit of
${rules.maxFiles} files reached`);
    else { seen.add(key); accepted.push(file); }
  }
  return { accepted, errors };
}
```

Being pure (no React, no DOM), it runs in plain Node tests with fake `{ name, type, size, lastModified }` objects.

Challenge 3: Drag & drop that does not flicker or open the file

Problem. Three classic bugs:

1. Without `preventDefault()` on `dragover`, the `drop` event **never fires**.
2. Without `preventDefault()` on `drop`, the browser navigates to the file (your app is gone).
3. Dragging over the zone's child (the button, the text) fires `dragleave` on the zone, then `dragenter` on the child → the highlight flickers on and off.

Solution. Prevent default on `dragover` and `drop`. Count enter/leave events in a ref and only turn the highlight off when the count gets back to 0.

```
const dragDepth = useRef(0);

const handleDragEnter = (e) => { e.preventDefault(); dragDepth.current += 1; setIsDragging(true); };
const handleDragLeave = (e) => { e.preventDefault(); dragDepth.current -= 1; if (dragDepth.current ===
0) setIsDragging(false); };
const handleDrop = (e) => { e.preventDefault(); dragDepth.current = 0; setIsDragging(false); if
(!disabled) onFiles(e.dataTransfer.files); };

<div onDragEnter={handleDragEnter} onDragLeave={handleDragLeave}
     onDragOver={(e) => e.preventDefault()} onDrop={handleDrop}>
```

(Alternative: CSS `pointer-events: none` on the children while dragging.)

Challenge 4: Picking the same file twice does nothing

Problem. The user picks `photo.png`, removes it, then picks `photo.png` again. The input's value did not change, so `onChange` **does not fire** and nothing happens.

Solution. Copy the files out first (the `FileList` belongs to the input), then reset the value.

```
const handleChange = (e) => {
  const files = Array.from(e.target.files);
  e.target.value = ''; // lets the user pick the same file again
  onFiles(files);
};
```

---

8. Implement in the correct order

#	Step	Working result
1	`<input type="file" multiple>` + `items` state + list of names	Picking files shows their names
2	`formatBytes`, `getKind`, show size per file	"photo.png · 3.5 KB"
3	`FilePreview` with object URL in effect + cleanup	Image thumbnails appear, URLs revoked on remove
4	Remove button + Clear all, stable `id` keys	Removing the right card, no wrong-media reuse
5	`RULES` + pure `validateFiles` (type, empty, size) + errors box	Bad files rejected with reasons
6	Duplicates + max count + reset input value	Same file blocked; same file re-pickable after remove
7	Extract `useFileSelection` hook	`App` is small; logic is reusable
8	`Dropzone`: hidden input + Browse button + drag events + depth counter	Drag & drop works, no flicker, no navigation

#	Step	Working result
9	Video + PDF previews (`Media` component)	All allowed kinds preview
10	Toolbar: count + total size (derived)	"2 files · 3.5 KB"
11	`uploadFiles` mock + status enum + disabled controls	Uploading / success / error states
12	A11y + CSS polish (labels, `role="alert"` , `aria-live` , focus ring)	Keyboard + screen reader friendly

9. Cover all states and edge cases

State / edge case	What the user sees / what happens
Initial / empty	Drop zone, hint, "No files selected yet."
Dragging over zone	Blue border + light blue background; stays on over children
Drag leaves zone	Highlight removed (depth counter back to 0)
Files selected	Grid of cards, toolbar with count and total size, Upload button
Image loading	Grey media box until `url` is set; `loading="lazy"` , `decoding="async"`
Invalid type / empty / too big	File skipped, reason listed in the red alert box
Mixed batch	Valid files added, invalid listed
Duplicate (across or within batch)	"already added"
Over max count	Fills up to 6, rest "limit of 6 files reached"
Same file re-picked after remove	Works (input value reset)
Unknown MIME (`type === ''`)	Rejected as "file type not allowed"
Long file name	Ellipsis + full name in `title` tooltip
Uploading	Button "Uploading…", drop/remove/clear disabled, drop ignored
Upload success	List cleared (URLs revoked), "Upload complete."
Upload error	Files kept, "Upload failed. Please try again."
New add after success/error	Status message cleared
Component unmount / leave page	Every card's cleanup revokes its URL
React StrictMode	Effects run twice in dev; create/revoke/create keeps it balanced

10. Final structure

```
_33_file_upload_preview/
└── src/
```

```
    ├── api/
    │   └── uploadFiles.js        # mock POST with FormData
    ├── components/
    │   ├── Dropzone.jsx          # drag & drop + hidden input
    │   └── FilePreview.jsx       # one card, owns the object URL
    ├── hooks/
    │   └── useFileSelection.js   # items, errors, add/remove/clear
    ├── utils/
    │   └── fileUtils.js          # RULES, validateFiles, formatBytes, getKind
    ├── App.jsx                   # container, upload status
    ├── App.css
    ├── index.css
    └── main.jsx                  # unchanged (StrictMode + createRoot)
```

11. Full working code

`src/utils/fileUtils.js`

```
export const RULES = {
  accept: ['image/', 'video/', 'application/pdf'],
  maxSize: 10 * 1024 * 1024, // 10 MB
  maxFiles: 6,
};

// "image/" -> "image/*" so the native picker filters the same way validation does
export const ACCEPT_ATTR = RULES.accept.map((rule) => (rule.endsWith('/') ? `${rule}*` :
rule)).join(',');

// name + size + lastModified identifies "the same file" picked twice
export const fileKey = (file) => `${file.name}-${file.size}-${file.lastModified}`;

export function formatBytes(bytes) {
  if (bytes < 1024) return `${bytes} B`;
  const units = ['KB', 'MB', 'GB'];
  let value = bytes / 1024;
  let i = 0;
  while (value >= 1024 && i < units.length - 1) {
    value /= 1024;
    i++;
  }
  return `${value.toFixed(1)} ${units[i]}`;
}

export function getKind(type) {
  if (type.startsWith('image/')) return 'image';
  if (type.startsWith('video/')) return 'video';
  if (type === 'application/pdf') return 'pdf';
  return 'file';
}

const isAccepted = (type, accept) =>
  accept.some((rule) => (rule.endsWith('/') ? type.startsWith(rule) : type === rule));

// Pure: splits incoming files into accepted ones and readable error messages.
export function validateFiles(incoming, existing, rules = RULES) {
```

```
  const seen = new Set(existing.map(fileKey));
  const accepted = [];
  const errors = [];

  for (const file of incoming) {
    const key = fileKey(file);
    if (!isAccepted(file.type, rules.accept)) errors.push(`${file.name}: file type not allowed`);
    else if (file.size === 0) errors.push(`${file.name}: file is empty`);
    else if (file.size > rules.maxSize) errors.push(`${file.name}: larger than
${formatBytes(rules.maxSize)}`);
    else if (seen.has(key)) errors.push(`${file.name}: already added`);
    else if (existing.length + accepted.length >= rules.maxFiles) errors.push(`${file.name}: limit of
${rules.maxFiles} files reached`);
    else {
      seen.add(key);
      accepted.push(file);
    }
  }
  return { accepted, errors };
}
```

src/api/uploadFiles.js

```
// Mock upload. A real app would do: fetch('/api/upload', { method: 'POST', body })
// (no Content-Type header: the browser sets the multipart boundary itself).
export function uploadFiles(files) {
  const body = new FormData();
  files.forEach((file) => body.append('files', file));

  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (Math.random() < 0.15) reject(new Error('Network error'));
      else resolve({ uploaded: files.length });
    }, 1200);
  });
}
```

src/hooks/useFileSelection.js

```
import { useState } from 'react';
import { RULES, getKind, validateFiles } from '../utils/fileUtils';

export function useFileSelection(rules = RULES) {
  const [items, setItems] = useState([]); // [{ id, file, kind }]
  const [errors, setErrors] = useState([]); // messages from the last add

  const addFiles = (fileList) => {
    const { accepted, errors } = validateFiles(Array.from(fileList), items.map((item) => item.file),
rules);
    setErrors(errors);
    if (accepted.length === 0) return;
    setItems((prev) => [
      ...prev,
      ...accepted.map((file) => ({ id: crypto.randomUUID(), file, kind: getKind(file.type) })),
    ]);
  };
```

```
  const removeFile = (id) => setItems((prev) => prev.filter((item) => item.id !== id));

  const clearAll = () => {
    setItems([]);
    setErrors([]);
  };

  return { items, errors, addFiles, removeFile, clearAll };
}
```

`src/components/Dropzone.jsx`

```
import { useRef, useState } from 'react';
import { ACCEPT_ATTR, RULES, formatBytes } from '../utils/fileUtils';

export default function Dropzone({ onFiles, disabled }) {
  const inputRef = useRef(null);
  const dragDepth = useRef(0); // dragenter/dragleave also fire for children: count them to avoid
flicker
  const [isDragging, setIsDragging] = useState(false);

  const handleChange = (e) => {
    const files = Array.from(e.target.files);
    e.target.value = ''; // lets the user pick the same file again
    onFiles(files);
  };

  const handleDragEnter = (e) => {
    e.preventDefault();
    dragDepth.current += 1;
    setIsDragging(true);
  };

  const handleDragLeave = (e) => {
    e.preventDefault();
    dragDepth.current -= 1;
    if (dragDepth.current === 0) setIsDragging(false);
  };

  const handleDrop = (e) => {
    e.preventDefault(); // otherwise the browser opens the file
    dragDepth.current = 0;
    setIsDragging(false);
    if (!disabled) onFiles(e.dataTransfer.files);
  };

  return (
    <div
      className={`dropzone${isDragging && !disabled ? ' dropzone--active' : ''}`}
      onDragEnter={handleDragEnter}
      onDragLeave={handleDragLeave}
      onDragOver={(e) => e.preventDefault()} // required, or drop never fires
      onDrop={handleDrop}
    >
      <p>Drag &amp;amp; drop files here, or</p>
      <button type="button" onClick={() => inputRef.current.click()} disabled={disabled} aria-
describedby="dropzone-hint">
```

```
        Browse files
      </button>
      <p id="dropzone-hint" className="hint">
        Images, videos or PDF · up to {formatBytes(RULES.maxSize)} each · max {RULES.maxFiles} files
      </p>
      <input ref={inputRef} type="file" multiple accept={ACCEPT_ATTR} onChange={handleChange} hidden
/>
    </div>
  );
}
```

`src/components/FilePreview.jsx`

```
import { useEffect, useState } from 'react';
import { formatBytes } from '../utils/fileUtils';

function Media({ kind, url, name }) {
  if (kind === 'image') return <img src={url} alt={name} loading="lazy" decoding="async" />;
  if (kind === 'video') return <video src={url} controls muted preload="metadata" />;
  if (kind === 'pdf') {
    return (
      <a href={url} target="_blank" rel="noreferrer" className="preview__icon">
        PDF · Open
      </a>
    );
  }
  return <span className="preview__icon">FILE</span>;
}

export default function FilePreview({ item, onRemove, disabled }) {
  const { file, kind } = item;
  const [url, setUrl] = useState(null);

  // The object URL lives exactly as long as this card: created on mount, revoked on unmount.
  useEffect(() => {
    const objectUrl = URL.createObjectURL(file);
    setUrl(objectUrl);
    return () => URL.revokeObjectURL(objectUrl);
  }, [file]);

  return (
    <li className="preview">
      <div className="preview__media">{url && <Media kind={kind} url={url} name={file.name} />}</div>
      <div className="preview__info">
        <span className="preview__name" title={file.name}>
          {file.name}
        </span>
        <span className="preview__size">{formatBytes(file.size)}</span>
      </div>
      <button
        type="button"
        className="preview__remove"
        onClick={() => onRemove(item.id)}
        disabled={disabled}
        aria-label={`Remove ${file.name}`}
      >
        ×
      </button>
```

```
    </li>
  );
}
```

`src/App.jsx`

```
import { useState } from 'react';
import Dropzone from './components/Dropzone';
import FilePreview from './components/FilePreview';
import { useFileSelection } from './hooks/useFileSelection';
import { uploadFiles } from './api/uploadFiles';
import { formatBytes } from './utils/fileUtils';
import './App.css';

const plural = (n) => `${n} file${n === 1 ? '' : 's'}`;

export default function App() {
  const { items, errors, addFiles, removeFile, clearAll } = useFileSelection();
  const [status, setStatus] = useState('idle'); // idle | uploading | success | error
  const isUploading = status === 'uploading';
  const totalSize = items.reduce((sum, item) => sum + item.file.size, 0); // derived, not stored

  const handleFiles = (files) => {
    setStatus('idle');
    addFiles(files);
  };

  async function handleUpload() {
    setStatus('uploading');
    try {
      await uploadFiles(items.map((item) => item.file));
      clearAll();
      setStatus('success');
    } catch {
      setStatus('error'); // keep the files so the user can retry
    }
  }

  return (
    <main className="app">
      <h1>File Upload Preview</h1>
      <Dropzone onFiles={handleFiles} disabled={isUploading} />

      {errors.length > 0 && (
        <div className="errors" role="alert">
          <ul>
            {errors.map((message, i) => (
              <li key={i}>{message}</li>
            ))}
          </ul>
        </div>
      )}

      {items.length === 0 ? (
        <p className="empty">No files selected yet.</p>
      ) : (
        <>
          <div className="toolbar">
```

```
            <span>
              {plural(items.length)} · {formatBytes(totalSize)}
            </span>
            <button type="button" onClick={clearAll} disabled={isUploading}>
              Clear all
            </button>
          </div>
          <ul className="grid">
            {items.map((item) => (
              <FilePreview key={item.id} item={item} onRemove={removeFile} disabled={isUploading} />
            ))}
          </ul>
          <button type="button" className="primary" onClick={handleUpload} disabled={isUploading}>
            {isUploading ? 'Uploading…' : `Upload ${plural(items.length)}`}
          </button>
        </>
      )}

      <p className="status" aria-live="polite">
        {status === 'success' && 'Upload complete.'}
        {status === 'error' && 'Upload failed. Please try again.'}
      </p>
    </main>
  );
}
```

`src/index.css`

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
body {
  margin: 0;
  font-family: system-ui, sans-serif;
  background: #f5f6f8;
  color: #1f2933;
}
button {
  font: inherit;
  cursor: pointer;
}
button:disabled {
  cursor: not-allowed;
  opacity: 0.6;
}
button:focus-visible {
  outline: 3px solid #93c5fd;
  outline-offset: 2px;
}
```

`src/App.css`

```
.app { max-width: 760px; margin: 40px auto; padding: 0 16px; }
.dropzone { border: 2px dashed #9aa5b1; border-radius: 12px; padding: 32px 16px; text-align: center;
```

```
background: #fff; transition: border-color 0.15s, background 0.15s; }
.dropzone--active { border-color: #2563eb; background: #eff6ff; }
.dropzone p { margin: 0 0 12px; }
.dropzone .hint { margin: 12px 0 0; font-size: 13px; color: #616e7c; }
.dropzone button, .primary { padding: 8px 16px; border: 0; border-radius: 8px; background: #2563eb;
color: #fff; }
.errors { margin-top: 16px; padding: 12px 16px; border-radius: 8px; background: #fef2f2; color:
#b91c1c; }
.errors ul { margin: 0; padding-left: 18px; }
.empty { margin: 24px 0; text-align: center; color: #616e7c; }
.toolbar { display: flex; justify-content: space-between; align-items: center; margin: 24px 0 12px; }
.toolbar button { padding: 4px 10px; border: 1px solid #cbd2d9; border-radius: 6px; background: none;
}
.grid { list-style: none; margin: 0 0 16px; padding: 0; display: grid; grid-template-columns:
repeat(auto-fill, minmax(160px, 1fr)); gap: 12px; }
.preview { position: relative; overflow: hidden; border-radius: 10px; background: #fff; box-shadow: 0
1px 3px rgba(0, 0, 0, 0.12); }
.preview__media { aspect-ratio: 4 / 3; display: grid; place-items: center; background: #e4e7eb; }
.preview__media img, .preview__media video { display: block; width: 100%; height: 100%; object-fit:
cover; }
.preview__icon { font-weight: 700; color: #52606d; text-decoration: none; }
.preview__info { display: flex; flex-direction: column; padding: 8px 10px; font-size: 13px; }
.preview__name { overflow: hidden; white-space: nowrap; text-overflow: ellipsis; font-weight: 500; }
.preview__size { color: #616e7c; }
.preview__remove { position: absolute; top: 6px; right: 6px; width: 28px; height: 28px; border: 0;
border-radius: 50%; background: rgba(0, 0, 0, 0.6); color: #fff; font-size: 18px; line-height: 1; }
.status { min-height: 1.5em; margin-top: 12px; }
```

---

12. Wrap-up

Edge cases handled

- Wrong type, empty file, too big, duplicates (also within one batch), max count, unknown MIME.
- Same file re-picked after removal (input reset).
- Drag flicker over children, drop navigating away, drop while uploading.
- Object URL leaks on remove, clear, successful upload, unmount and StrictMode.
- Failed upload keeps the files; controls disabled while uploading.

Trade-offs to mention

- **Object URL vs FileReader:** object URL is instant and has no base64 copy, but must be revoked. FileReader gives a data URL you could persist (e.g. `localStorage`), at a big memory/time cost.
- **URL in the card's effect vs in state:** one extra render (empty box, then image) in exchange for cleanup that cannot be forgotten.
- **Validation by MIME type:** `file.type` comes from the file extension, so a renamed `.exe → .png` passes. Real safety is a **server-side** check (magic bytes, virus scan). Client validation is UX only.
- **Partial accept vs reject batch:** partial accept is friendlier; rejecting all is simpler to explain. We chose friendly, with per-file reasons.
- **Errors replaced on each add:** the user sees only the latest problems, not a growing log.
- **One request for all files:** simple, but one failure fails everything and there is no per-file progress.

- **PDF as a link, not an embed:** embedding 6 PDF viewers is heavy; a link is cheap and works everywhere.

What to add with more time

- **Per-file upload with progress and retry** (`XMLHttpRequest.upload.onprogress` or chunked upload), `AbortController` to cancel (task _41).
- **Real thumbnails for big images**: `createImageBitmap` + canvas to downscale, so 20 MP photos do not decode at full size.
- **Image dimensions / video duration** validation (load metadata first).
- **Reorder** cards by drag, **paste** from clipboard (`onPaste` → `clipboardData.files`).
- **Window-level drop guard** so dropping outside the zone does not open the file.
- **Tests:** unit tests for `validateFiles` / `formatBytes` ; React Testing Library for add, remove, duplicate, upload states (`userEvent.upload`).
- **Virtualize** the grid if hundreds of files are allowed.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why do you still validate when the input already has `accept="image/*"` ?

`accept` is only a hint for the native picker. The user can choose "All files", and drag & drop ignores it completely. Validation must run on every path that adds files, so it lives in `addFiles` , not in the input.

Q2. Is client-side validation enough for security?

No. `file.type` is guessed from the extension, so `virus.exe` renamed to `virus.png` passes. Client checks are for UX (fast feedback). The server must check size, real type (magic bytes), and scan the file.

Q3. What if the interviewer wants to reject the whole batch if one file is invalid?

`validateFiles` already returns both lists. In `addFiles` , only call `setItems` when `errors.length === 0` . The change is one line because the decision logic is pure and separate.

State design

Q4. Why not store the preview URL in the items state?

Then three places (remove, clear, unmount) must remember to revoke it, and a missed one leaks memory. Putting it in the card's effect ties the URL to the card's lifetime, so React's cleanup does it for us.

Q5. Why `crypto.randomUUID()` for the id and not the index or file name?

Indexes shift when an item is removed, so React would reuse the wrong `<video>` element (wrong playback state, wrong image flash). Names are not unique. A generated id is stable for the item's whole life.

Q6. Why is `totalSize` not in state?

It is fully derivable from `items`. Storing it means updating it in add, remove and clear, and one day it will be wrong. `reduce` over at most 6 items costs nothing.

Q7. Why a `status` enum instead of `isUploading`, `isSuccess`, `isError`?

Booleans allow impossible combinations (uploading **and** error). One enum allows exactly one state at a time and makes the UI a simple switch.

Hard challenges

Q8. `FileReader.readAsDataURL` vs `URL.createObjectURL`?

FileReader reads the whole file into a base64 string: async, ~33% bigger, slow for videos, but serializable. `createObjectURL` returns a short `blob:` URL instantly that points to the file; it must be revoked. For previews, object URLs win.

Q9. What happens in React StrictMode with your object URL effect?

In development React mounts, unmounts and mounts again. Our effect creates a URL, the cleanup revokes it, then a new one is created. Nothing leaks and the `<img>` uses the final URL. A `useMemo` version would leak one URL.

Q10. Why does the drop zone flicker during drag, and how did you fix it?

`dragleave` fires on the zone when the pointer moves onto a child element, followed by `dragenter` on the child. We count enters and leaves in a ref and only clear the highlight when the counter returns to 0.

Q11. Why must you call `preventDefault()` on `dragover`?

By default most elements are not drop targets. Cancelling `dragover` tells the browser "this element accepts the drop"; without it `drop` never fires. On `drop` we prevent default too, or the browser opens the file and the app is gone.

Q12. Picking the same file twice does nothing. Why?

The input's value is unchanged, so no `change` event. We copy the files with `Array.from` and then set `e.target.value = ''`, so the next pick of the same file is a change.

Implementation steps

Q13. Why is the file input `hidden` with a separate button?

The native input is hard to style. A real `<button>` keeps keyboard and screen-reader support and calls `inputRef.current.click()`. Programmatic `click()` works on a hidden input because it runs inside a user gesture.

Q14. Why `Array.from(fileList)` everywhere?

`FileList` is array-like, not an array: no `map` / `filter`. And it belongs to the input; resetting the input's value can replace it. Copying to a real array first makes the data ours.

Q15. Why don't you set `Content-Type: multipart/form-data` in the fetch call?

With `FormData` as the body, the browser sets `Content-Type` **with the boundary** string. If you set it yourself you lose the boundary and the server cannot parse the body.

"What if" twists

Q16. What if users drop 500 files or 200 MB images?

Validation still runs in one pass (cheap). For the UI: virtualize the grid, generate small thumbnails with `createImageBitmap(file, { resizeWidth: 320 })` + canvas (ideally in a Web Worker), and only create object URLs for visible cards.

Q17. What if the API is slow or fails halfway?

Upload each file separately (with a concurrency limit, e.g. 3), keep per-file `status` and `progress` on each item, add retry for failed ones and `AbortController` to cancel. For very large files, chunked / resumable uploads.

Q18. What if they want to drop a whole folder?

`dataTransfer.files` only gives top-level files. Use `dataTransfer.items[i].webkitGetAsEntry()` and walk directories recursively, or `<input webkitdirectory>` for the picker.

Q19. What if the user should be able to paste a screenshot?

Listen to `paste` on the window (or zone) and pass `e.clipboardData.files` to the same `addFiles`. All validation and previews are reused.

Q20. What if the user accidentally closes the tab with files selected?

Add a `beforeunload` listener while `items.length > 0` and not uploaded. Files themselves cannot be restored after reload (a `File` cannot be put in `localStorage`); IndexedDB can store Blobs if that is a real requirement.

Trade-offs / "why not X"

Q21. Why not a library like `react-dropzone`?

In production it is a fine choice (handles folders, edge cases, accessibility). In an interview the goal is to show you understand the DOM events, `FileList`, object URLs and validation. The hand-written version is ~50 lines.

Q22. Why not embed PDFs with `<iframe>`?

Each iframe starts a full PDF viewer; 6 of them make the page heavy and slow. A link that opens the blob URL in a new tab gives a full viewer on demand. With more time: render page 1 to a canvas with `pdf.js` as a thumbnail.

Q23. Why are errors replaced on every add instead of accumulated?

The user cares about what just went wrong. Accumulating creates a growing, stale log that no longer matches the list (e.g. "limit reached" after they removed files). Replacing keeps the message relevant.